UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
Facultad de Ciencias 2026
Introducción a Ciencias de la Computación

REPORTE — PRÁCTICA 5

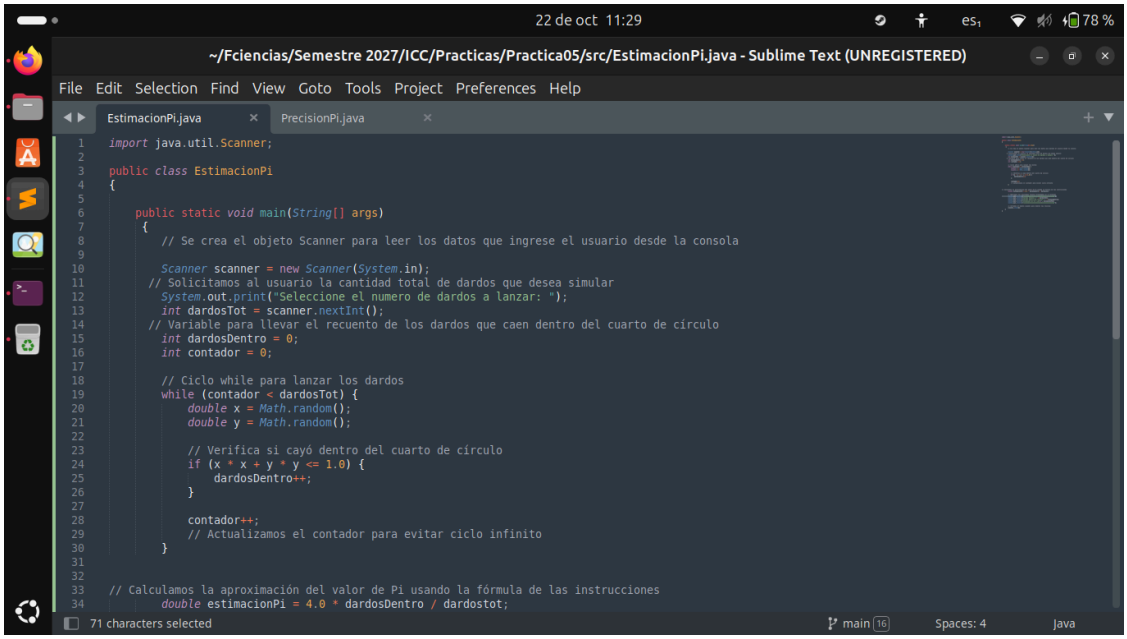
Decisiones y repeticiones

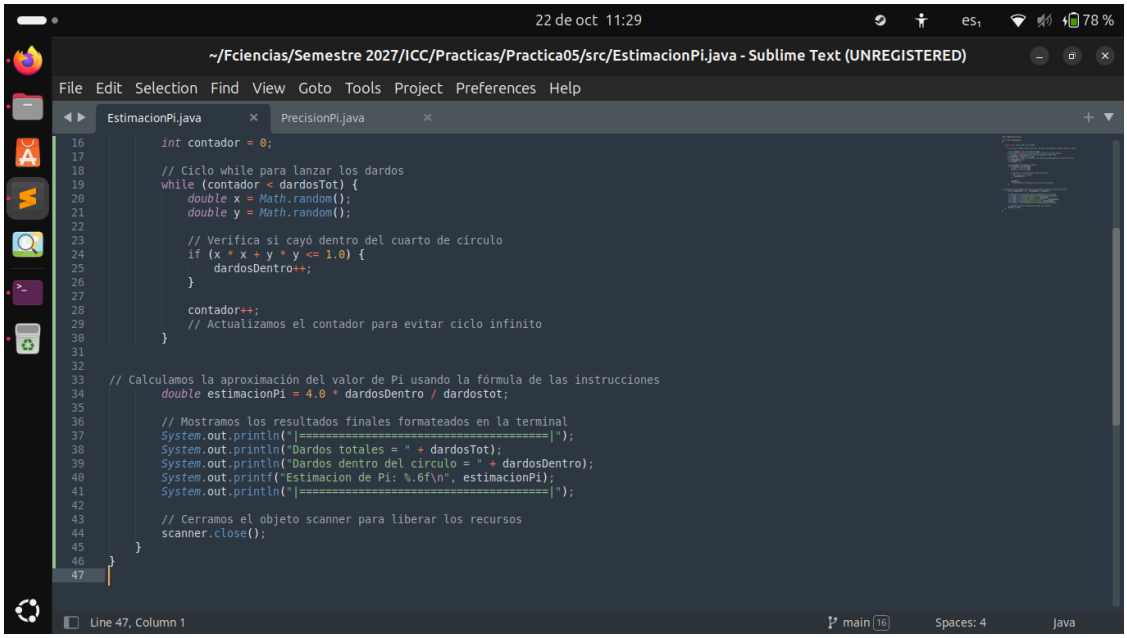
Nombre: López Pérez Luis Ángel
Número de cuenta: 324243174
Sistema operativo: Linux
Distribución: Ubuntu
Fecha: **22 / 10 / 2026**

Parte 1: Calculando π a dardazos

Actividad 1.1 – Un número fijo de dardos (EstimacionPi.java)

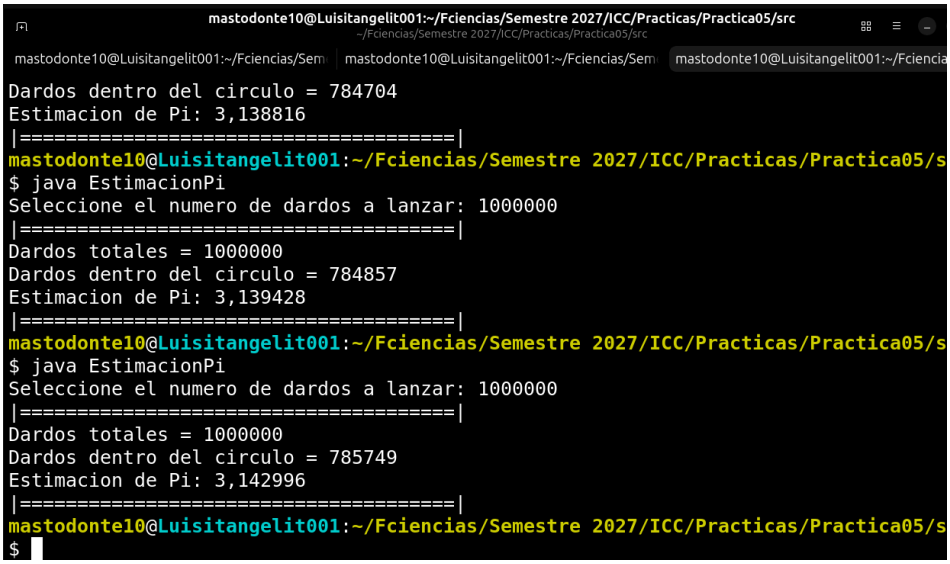
Para evitar modificar el programa, compilarlo y ejecutarlo 45 veces hacemos que el programa nos pida con Scanner cuántos dardos lanzar y lo guarde en dardosTot, usando dos contadores enteros: contador, que cuenta los dardos lanzados, y dardosDentro, que cuenta los que caen en el cuarto de círculo. Un ciclo while (contador < dardosTot) genera en cada vuelta dos coordenadas aleatorias x y y con Math.random(), ambas en el intervalo [0, 1). Con ayuda de la forma dada y un if reviso si x * x + y * y <= 1.0; si se cumple, el dardo cayó dentro y aumento dardosDentro. Al final de cada vuelta incremento contador, porque si no el ciclo nunca terminaría. Cuando el ciclo acaba, calcula $\pi \approx 4.0 \times$ dardosDentro / dardosTot, usando 4.0 para que la división sea con decimales y no entera, e imprimo los dardos totales, los que cayeron dentro y la estimación con seis decimales (%.6f).





Posteriormente ejecuté el programa tres veces con cada cantidad de dardos.

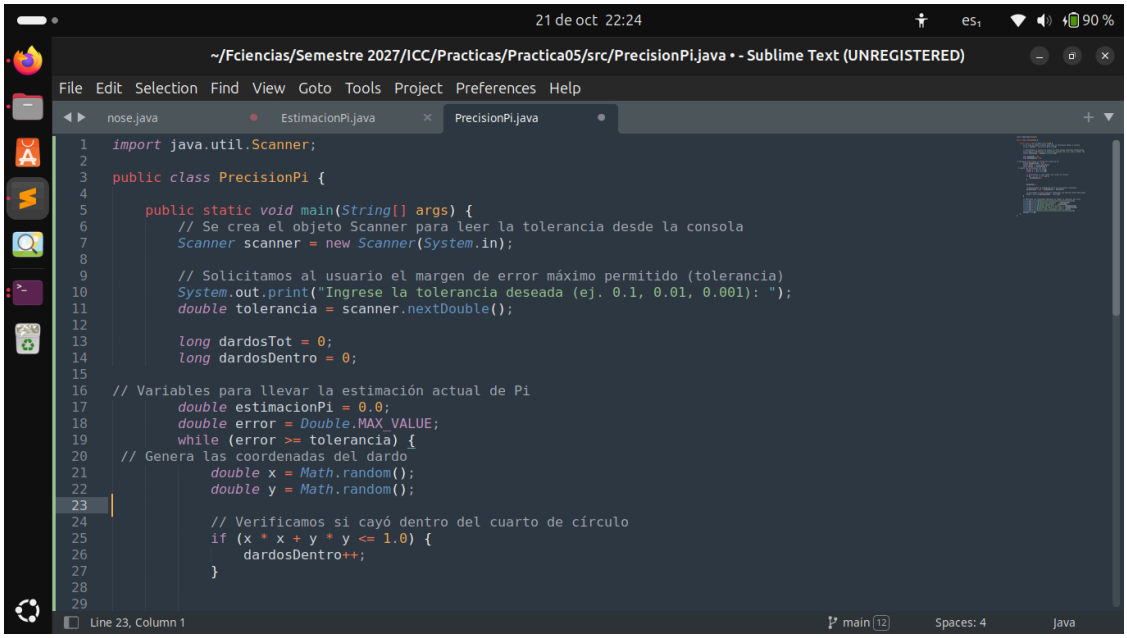
Num. de experimento	Número de dardos	Estimación de π
1	100	2.76
2	100	3.24
3	100	2.97
4	1,000	3.152
5	1,000	3.104
6	1,000	3.147
7	10,000	3.1124
8	10,000	3.1537
9	10,000	3.1378
10	100,000	3.144840
11	100,000	3,141682
12	100,000	3.142120
13	1,000,000	3.138816
14	1,000,000	3.139428
15	1,000,000	3.142996

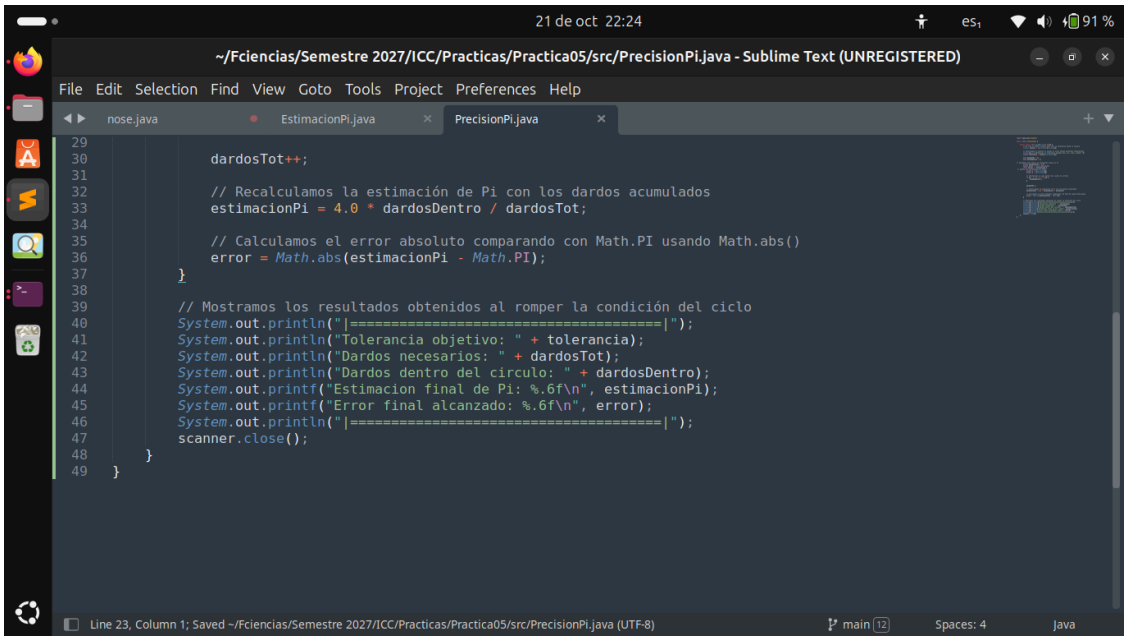


Comentarios. Conforme aumenta el número de dardos, las estimaciones se acercan a π (3.141593) y, además, se parecen más entre sí. Con 100 dardos la variación es grande, porque unos pocos dardos de más o de menos dentro del círculo cambian bastante el resultado. Con 1,000,000 de dardos esos desbalances se promedian y los valores quedan muy cerca del valor real.

Actividad 1.2 – Repetir hasta llegar a la precisión deseada (PrecisionPi.java)

Estrategia. Aquí le doy la vuelta a la pregunta: en lugar de fijar cuántos dardos lanzar, fijo el error máximo que acepto. El programa lee la tolerancia con Scanner y mantiene tres variables: dardosTot, dardosDentro y error. Como el error todavía no existe antes del primer dardo, lo inicializo con Double.MAX_VALUE, un valor enorme que garantiza que la condición while sea verdadera y el ciclo arranque. En cada vuelta genera un dardo con Math.random(), reviso con if si cae dentro (x * x + y * y <= 1.0), aumenta dardosTot y calcula la estimación 4.0 × dardosDentro / dardosTot con todos los dardos acumulados. Luego actualiza el error como Math.abs(estimacionPi - Math.PI); uso Math.abs porque el error es una distancia y no interesa si la estimación queda por arriba o por debajo de π. El ciclo termina cuando el error baja de la tolerancia, y entonces imprime los datos necesarios.

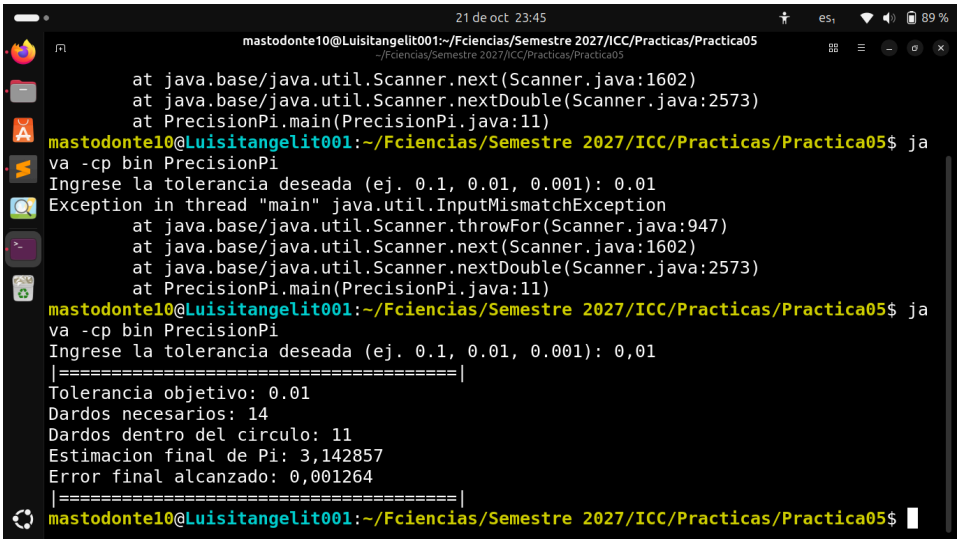


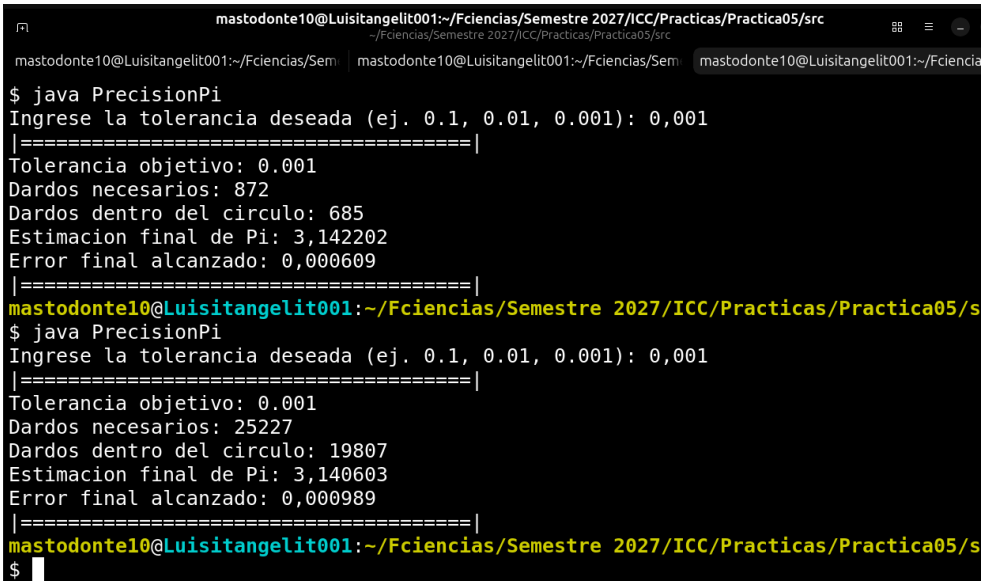


Resultados. Ejecuté el programa tres veces con cada tolerancia. La primera fila de 0.01 corresponde a la ejecución de la captura.

Tolerancia	Ejecución	Dardos necesarios	Estimación de π
0.1	1	66	3.212121
0.1	2	276	3.237537
0.1	3	13	3.076923

Tolerancia	Ejecución	Dardos necesarios	Estimación de π
0.1	**Promedio**	**118.3333…**	**3.175527**
0.01	1	94	3.142857
0.01	2	88	3.142345
0.01	3	149	3.136364
0.01	**Promedio**	**110.33**	**3.140522**
0.001	1	2598	3.1412451
0.001	2	872	3.142202
0.001	3	25227 xd	3.1406603
0.001	**Promedio**	**9565.67 xd**	**3.141369**





Preguntas.

¿Qué tanto variaron los dardos necesarios entre las tres ejecuciones de una misma tolerancia? ¿El promedio representa bien lo que pasó? Al final la run termina con un máximo de 25227 dardos con tolerancia e ironicamente el numero 11 de las 3 ejecuciones se acercó muchisimo mas que cualquier otro, en general la variación es muy grande, y crece al reducir la tolerancia, porque el resultado depende del azar: una run puede cruzar la tolerancia con pocos dardos y otra necesitar muchísimos más. **-Comparación con la actividad 1.1.** No concuerden casi nada, con un número fijo de dardos, un error menor a 0.001 casi siempre toma cientos o miles, mientras que en esta actividad el programa a veces se detiene mucho antes. La razón es el momento exacto en que se detiene el ciclo: se detiene la primera vez que la estimación cae dentro de la tolerancia, aunque sea por casualidad y con pocos dardos. Ese resultado es una coincidencia afortunada, no una estimación estable; si siguiera lanzando dardos, la estimación podría volver a alejarse y salir de la tolerancia. En 1.1 el resultado se mide siempre al final de un número fijo de dardos, por lo que refleja mejor el comportamiento real del método.

Parte 2: El regreso del señor Pines

Actividad 2.1 – Verificar los datos y el tipo de cliente (CotizadorPines.java)

Datos que valido y cómo. Partí de los datos libres del cotizador de la práctica 4. El programa lee el préstamo, el plazo en meses, la tasa de interés anual base y el tipo de cliente, y antes de hacer cualquier cuenta los revisa con una cadena de if / else if. Si un dato no tiene sentido, imprime un mensaje de error que explica el problema, cierra el Scanner y termina con return, de modo que no se calcula nada con datos inválidos. Las revisiones son:

- El monto del préstamo debe ser mayor que cero (prestamo <= 0 es error).
- El plazo en meses debe ser mayor que cero (plazoMeses <= 0 es error), lo que descarta casos como el plazo de −5 meses de la carta del señor Pines.
- La tasa de interés anual debe ser mayor que cero (tasaBase <= 0 es error).
- El tipo de cliente solo puede ser P, E o R; cualquier otra letra se rechaza.

Elegí estas condiciones porque un préstamo, un plazo o una tasa negativos o iguales a cero no representan una operación real y producirían cálculos absurdos. La lectura del tipo de cliente pasa por toUpperCase(), así que también acepta minúsculas.

Estrategia para la tasa según el tipo de cliente. Con los datos ya validados uso una estructura if / else if / else sobre tipoCliente para calcular la tasa aplicada a partir de la tasa base: si es P (Preferente), resto 2 puntos; si es E (Estándar), la tasa queda igual; y si es R (Riesgoso), le sumo 3 puntos. Con la tasa aplicada calculo el interés simple (capital × tasa × plazo en años) y el total a pagar, igual que en la práctica anterior.

Resultados de las cinco ejecuciones. Hice tres ejecuciones con los datos de Robbie cambiando el tipo de cliente entre P, E y R, y dos ejecuciones con datos inválidos para comprobar que el programa los detecta.

De las cuales los resultados fueron los siguientes:

1) Cliente P

Datos ingresados:

Prestamo: 12899.0

Meses: 21

Tasa base: 15.0

Tipo de cliente: P

Resultados:

Tasa aplicada: 13.00%

Interes simple: $2934.52

Total con interes simple: $15833.52

Interes compuesto (Ciclo): $3293.83

Total con interes compuesto (Ciclo): $16192.83

Total con interes compuesto (Formula Math.pow): $16192.83

2) Cliente E

Datos ingresados:

Prestamo: 12899.0

Meses: 21

Tasa base: 15.0

Tipo de cliente: E

Resultados:

Tasa aplicada: 15.00%

Interes simple: $3385.99

Total con interes simple: $16284.99

Interes compuesto (Ciclo): $3844.71

Total con interes compuesto (Ciclo): $16743.71

Total con interes compuesto (Formula Math.pow): $16743.71

3) Cliente R

Datos ingresados:

Prestamo: 12899.0

Meses: 21

Tasa base: 15.0

Tipo de cliente: R

Resultados:

Tasa aplicada: 18.00%

Interes simple: $4063.19

Total con interes simple: $16962.19

Interes compuesto (Ciclo): $4732.11

Total con interes compuesto (Ciclo): $17631.11

Total con interes compuesto (Formula Math.pow): $17631.11

4) Invalido por plazo negativo

Datos ingresados:

Prestamo: 12899.0

Meses: -5

Tasa base: 15.0

Tipo de cliente: E

Salida del programa:

[ERROR] El plazo en meses debe ser mayor a cero.

5) Dato invalido

Datos ingresados:

Prestamo: 12899.0

Meses: 21

Tasa base: 15.0

Tipo de cliente: X

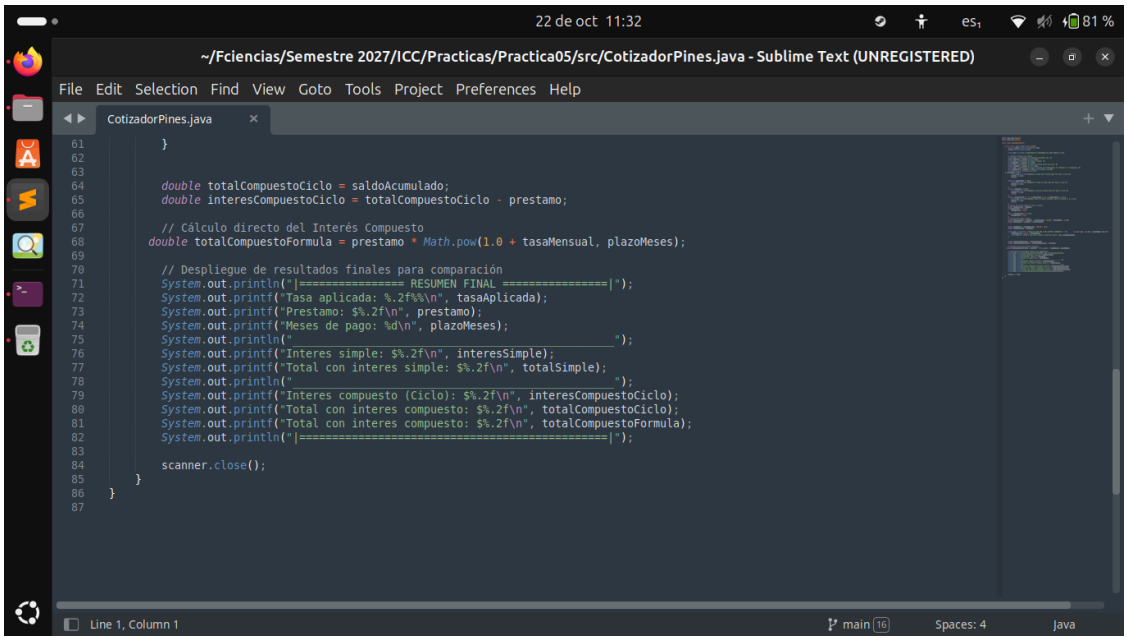
Salida del programa:

[ERROR] Tipo de cliente invalido. Solo se permite P, E o R.

Actividad 2.2 – Interés compuesto

Estrategia del For Primero uso la formula que nos dieron (tasaMensual = tasaAplicada / 100 / 12). El saldo acumulado empieza igual al capital. Después uso un ciclo for que da una vuelta por cada mes del plazo; en cada vuelta multiplico el saldo por (1 + tasaMensual) e imprimo un renglón con el número de mes y el saldo, con el formato "En el mes X se tiene un saldo de …". Elegí for y no while porque sé de antemano cuántas vueltas necesito: exactamente **(el profesor lo habia mencionado)** plazoMeses, con el contador integrado en el propio ciclo. Cuando el ciclo termina, el saldo acumulado es el total con interés compuesto, y el interés compuesto es ese total menos el capital.

Cálculo con la fórmula. Como multiplicar una vez por mes equivale a elevar a una potencia, también calculo directamente totalCompuestoFormula = prestamo * Math.pow(1.0 + tasaMensual, plazoMeses). El programa imprime ambos resultados, el del ciclo y el de la fórmula, para compararlos.




```
        }

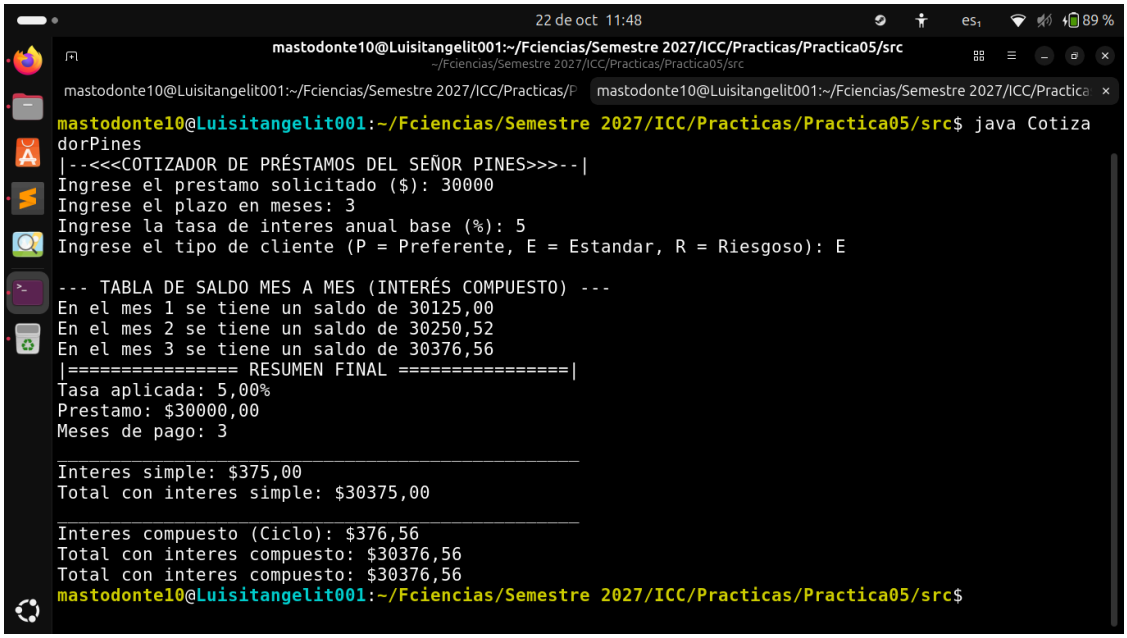

        double totalCompuestoCiclo = saldoAcumulado;
        double interesCompuestoCiclo = totalCompuestoCiclo - prestamo;

        // Cálculo directo del Interés Compuesto
      double totalCompuestoFormula = prestamo * Math.pow(1.0 + tasaMensual, plazoMeses);

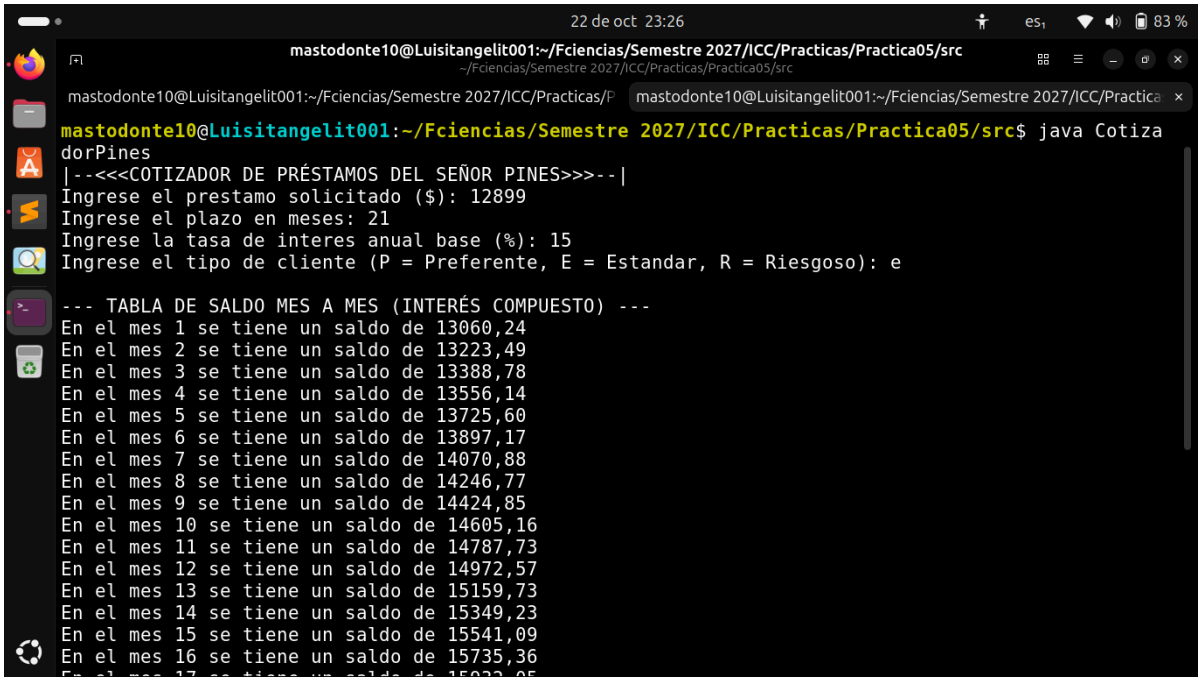
        // Despliegue de resultados finales para comparación
        System.out.println("|=============== RESUMEN FINAL ===============|");
        System.out.printf("Tasa aplicada: %.2f%%\n", tasaAplicada);
        System.out.printf("Prestamo: $%.2f\n", prestamo);
        System.out.printf("Meses de pago: %d\n", plazoMeses);
        System.out.println("_____________________________________________");
        System.out.printf("Interes simple: $%.2f\n", interesSimple);
        System.out.printf("Total con interes simple: $%.2f\n", totalSimple);
        System.out.println("_____________________________________________");
        System.out.printf("Interes compuesto (Ciclo): $%.2f\n", interesCompuestoCiclo);
        System.out.printf("Total con interes compuesto: $%.2f\n", totalCompuestoCiclo);
        System.out.printf("Total con interes compuesto: $%.2f\n", totalCompuestoFormula);
        System.out.println("|=============================================|");

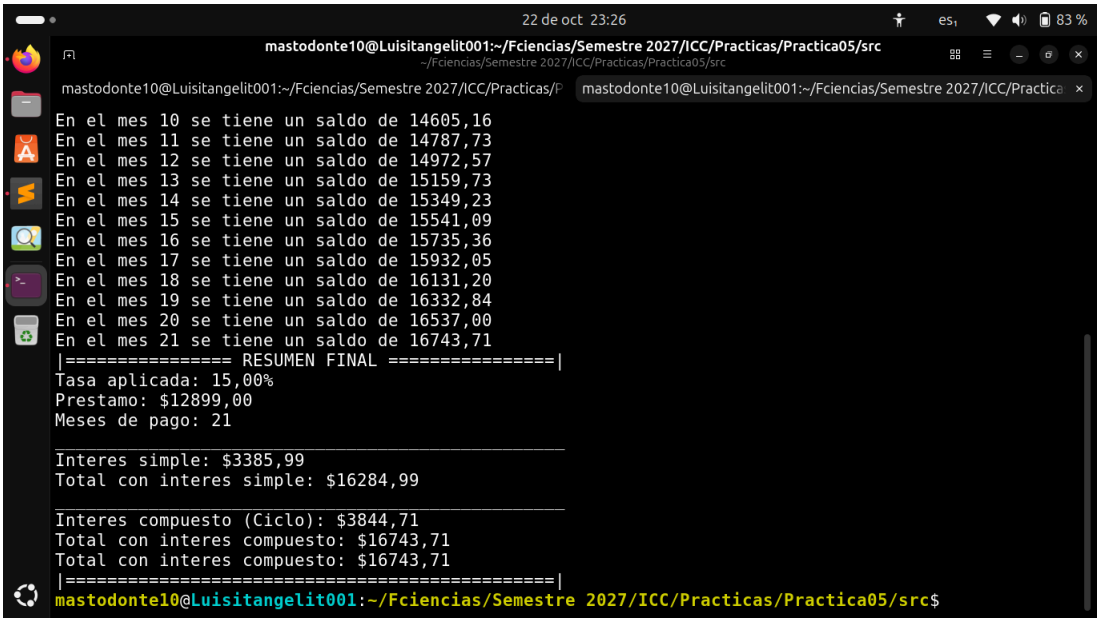
        scanner.close();
    }
}
```

Ejecución de prueba. Con un préstamo de $30,000, 3 meses, tasa de 5 % y cliente estándar, el programa muestra los saldos de cada mes ($30,125.00, $30,250.52 y $30,376.56) y en el resumen final el total con interés compuesto del ciclo y el de la fórmula coinciden en $30,376.56, con un interés compuesto de $376.56 frente a un interés simple de $375.00. El interés compuesto resulta mayor porque cada mes los intereses también generan intereses.



Verificación con los datos de Robbie. Los valores esperados son: tasa aplicada de 15 %, préstamo de $12,899.00, a 21 meses, interés simple de $3,385.99, total con interés simple de $16,284.99, interés compuesto de $3,844.71 y total con interés compuesto de $16,743.71.





Al mostrarse los valores de igual manera que en las instrucciones y ejecutarse correctamente se finaliza la práctica.

Conclusiones

En esta práctica pasé de programas que se ejecutan una sola vez, línea por línea, a programas que deciden y repiten. Con while programé los dos métodos Monte Carlo para aproximar π, y con for el interés compuesto del cotizador, que da una vuelta por cada mes. Con if, else if y else validé los datos y ajusté la tasa según el tipo de cliente. También usé herramientas que ya trae Java: Math.random(), Math.abs(), Math.PI y Math.pow().

Lo que más me llamó la atención fue ver, en las dos actividades de π la 11 a pesar de no ser tan grande el número se acerco demasiado a π